Project Design Phase-II
Solution Requirements (Functional & Non-functional)

Date	08 March 2026
Team ID	
Project Name	Ucab – Cab Booking System
Maximum Marks	4 Marks

Functional Requirements:

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	User Registration	Registration through Form
FR-2	User Confirmation	Confirmation via Notification
FR-3	Ride Booking	Select pickup location on interactive map Select drop-off location on interactive map Choose cab type (Economy / Premium / Shared) View estimated fare and arrival time before confirming Confirm and submit ride booking request
FR-4	Real-Time Ride Tracking	View assigned driver details after booking confirmation Track driver location live on map from pickup to drop-off Receive real-time status updates (Driver on the way / Ride started / Completed) Automatic ride status phase transitions via Socket.IO
	Driver Dashboard	View and accept pending ride requests in real time Receive new ride request notifications via Socket.IO Mark ride as started and completed View completed ride history and earnings summary
	Payment & Support	Make digital payment via saved payment method after ride Automatically generate and view payment receipt Rate driver (1–5 stars) after ride completion Raise a support ticket for any ride-related issue View all past bookings and payment history anytime

Non-functional Requirements:

Following are the non-functional requirements of the proposed solution.

FR No.	Non-Functional Requirement	Description
NFR-1	**Usability**	The UCab interface is built with React.js and Bootstrap ensuring a clean, responsive layout across all devices and screen sizes. Intuitive navigation with role-specific dashboards (Rider / Driver) minimises

		user learning curve. Booking flow (select location → confirm → track → pay → rate) is completed in under 5 steps.
NFR-2	**Security**	All user passwords are hashed using bcryptjs before storage. API routes are protected with JWT (JSON Web Token) bearer authentication. Role-based route guards ensure Riders cannot access Driver/Admin routes and vice versa. CORS is configured to restrict cross-origin access. Sensitive credentials (JWT secret, DB URI) are stored in .env files and never exposed to the frontend.
NFR-3	**Reliability**	The system uses Mongoose schema validation to enforce data integrity at the database layer. Express middleware handles errors gracefully and returns structured JSON error responses. Socket.IO connections include reconnection logic to handle temporary network drops. MongoDB Atlas (cloud) provides automated backups and high-availability replication.
NFR-4	**Performance**	MongoDB queries are optimised with indexing on frequently accessed fields (userId, driverId, bookingStatus). Axios is used on the frontend for async, non-blocking API calls. Socket.IO room-based event broadcasting limits event delivery to relevant clients only, reducing unnecessary network load. React component-level state management avoids full-page re-renders for real-time updates.
NFR-5	**Availability**	The backend is designed for deployment on cloud platforms (AWS / Azure / Heroku) with environment-based configuration. The application supports 99.9% uptime targets using managed MongoDB Atlas clusters with automatic failover. Frontend is served as a static build, deployable on CDN-backed services (Netlify / Vercel) for high availability.
NFR-6	**Scalability**	The modular Express.js backend (auth, bookings, drivers, payments, support routes) can be independently scaled or extracted into microservices. MongoDB's horizontal scaling (sharding) supports data growth without schema migration. Socket.IO's room architecture scales to multiple cities and vehicle types by adding new rooms. The React frontend's component model allows rapid feature addition without architectural changes.